

Tóm tắt

Các mô hình chuyển đổi chuỗi chỉ phối hiện nay dựa trên các mạng nơ-ron hồi quy hoặc mạng nơ-ron tích chập phức tạp, bao gồm một bộ mã hóa và một bộ giải mã. Các mô hình có hiệu suất tốt nhất cũng kết nối bộ mã hóa và bộ giải mã thông qua một cơ chế chú ý. Chúng tôi đề xuất một kiến trúc mạng mới đơn giản, Transformer, dựa hoàn toàn trên các cơ chế chú ý, loại bỏ hoàn toàn việc sử dụng hồi quy và tích chập. Các thí nghiệm trên hai nhiệm vụ dịch máy cho thấy các mô hình này vượt trội về chất lượng đồng thời có khả năng song song cao hơn và yêu cầu thời gian huấn luyện đáng kể ít hơn. Mô hình của chúng tôi đạt 28.4 BLEU trên nhiệm vụ dịch WMT 2014 Tiếng Anh-Tiếng Đức, cải thiện hơn 2 BLEU so với các kết quả tốt nhất hiện có, kể cả các mô hình hợp nhất. Trên nhiệm vụ dịch WMT 2014 Tiếng Anh-Tiếng Pháp, mô hình của chúng tôi thiết lập điểm BLEU mới nhất cho mô hình đơn là 41.8 sau 3.5 ngày huấn luyện trên tám GPU, chỉ chiếm một phần rất nhỏ chi phí huấn luyện so với các mô hình tốt nhất trong tài liệu. Chúng tôi chứng minh rằng Transformer tổng quát tốt cho các nhiệm vụ khác bằng cách áp dụng thành công cho việc phân tích cú pháp thành phần tiếng Anh cả với dữ liệu huấn luyện lớn và hạn chế.

** Đóng góp bằng nhau. Thứ tự liệt kê là ngẫu nhiên. Jakob đề xuất thay thế các mạng nơ-ron hồi quy bằng tự chú ý và khởi xướng nỗ lực đánh giá ý tưởng này. Ashish, cùng Illia, thiết kế và triển khai các mô hình Transformer đầu tiên và đã tham gia quan trọng vào mọi khía cạnh của công trình này. Noam đề xuất chú ý tích vô hướng tỷ lệ, chú ý đa đầu và biểu diễn vị trí không tham số, và trở thành người tham gia vào hầu hết mọi chi tiết. Niki thiết kế, triển khai, tinh chỉnh và đánh giá vô số biến thể mô hình trong cơ sở mã gốc và tensor2tensor của chúng tôi. Llion cũng thực nghiệm với các biến thể mô hình mới, chịu trách nhiệm cho cơ sở mã ban đầu của chúng tôi, cũng như việc suy luận hiệu quả và trực quan hoá. Lukasz và Aidan đã dành nhiều ngày dài thiết kế các phần khác nhau và triển khai tensor2tensor, thay thế cơ sở mã trước đây, cải thiện đáng kể kết quả và tăng tốc mạnh mẽ nghiên cứu của chúng tôi.*

† Công việc thực hiện trong thời gian làm việc tại Google Brain.

‡ Công việc thực hiện trong thời gian làm việc tại Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

1 Giới thiệu

Các mạng nơ-ron hồi quy, mạng bộ nhớ dài-ngắn hạn [13] và các mạng nơ-ron hồi quy có cổng [7] nói riêng, đã được khẳng định vững chắc là các phương pháp tiên tiến nhất trong các vấn đề mô hình hoá và chuyển đổi chuỗi như mô hình hoá ngôn ngữ và dịch máy [35, 2, 5]. Nhiều nỗ lực sau này tiếp tục đẩy giới hạn của các mô hình ngôn ngữ hồi quy và kiến trúc bộ mã hóa-bộ giải mã [38, 24, 15].

Các mô hình hồi quy thường phân chia tính toán dọc theo các vị trí ký hiệu của chuỗi đầu vào và đầu ra. Khi đồng bộ các vị trí với các bước thời gian tính toán, chúng tạo ra một chuỗi các trạng thái ẩn ht , như một hàm của trạng thái ẩn trước $ht-1$ và đầu vào tại vị trí t . Tính chất tuần tự nội tại này ngăn cản việc song song hoá trong các ví dụ huấn luyện, điều này trở nên quan trọng khi độ dài chuỗi tăng, vì các giới hạn bộ nhớ hạn chế việc tạo batch qua các ví dụ. Các công trình gần đây đã đạt được cải thiện đáng kể về hiệu quả tính toán thông qua các thủ thuật phân rã [21] và tính toán có điều kiện [32], đồng thời cũng nâng cao hiệu suất mô hình trong trường hợp sau. Tuy nhiên, ràng buộc cơ bản của tính toán tuần tự vẫn còn tồn tại.

Các cơ chế chú ý đã trở thành một phần không thể thiếu của các mô hình mô hình hoá và chuyển đổi chuỗi ẩn tượng trong nhiều nhiệm vụ, cho phép mô hình hoá các phụ thuộc mà không quan tâm đến khoảng cách của chúng trong chuỗi đầu vào hoặc đầu ra [2, 19]. Trong hầu hết các trường hợp [27], các cơ chế chú ý này vẫn được sử dụng kết hợp với một mạng hồi quy.

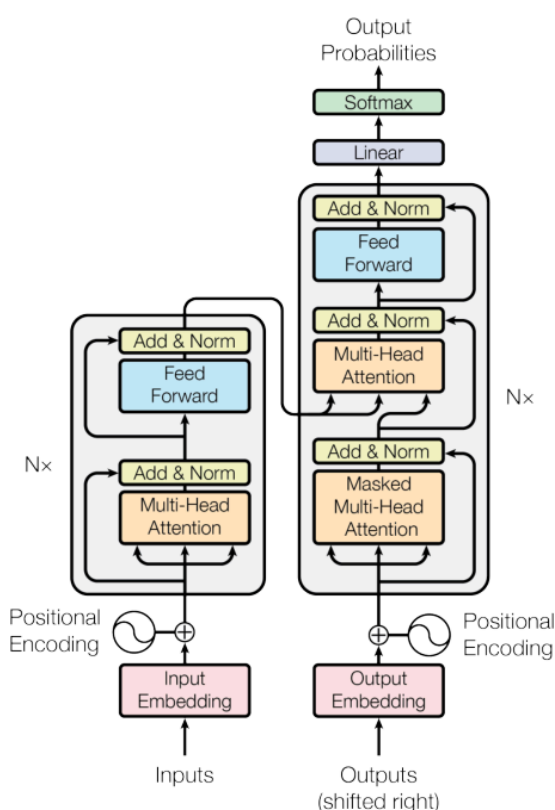
Trong công trình này, chúng tôi đề xuất Transformer, một kiến trúc mô hình loại bỏ hoàn toàn việc hồi quy và thay vào đó dựa toàn bộ vào một cơ chế chú ý để tạo ra các phụ thuộc toàn cục giữa đầu vào và đầu ra. Transformer cho phép song song hoá đáng kể hơn và có thể đạt được trạng thái tiên tiến mới về chất lượng dịch sau khi được huấn luyện chỉ trong vòng mười hai giờ trên tám GPU P100.

2 Cơ sở

Mục tiêu giảm tính toán tuần tự cũng là nền tảng của Extended Neural GPU [16], ByteNet [18] và ConvS2S [9], tất cả đều sử dụng mạng nơ-ron tích chập làm khối xây dựng cơ bản, tính toán các biểu diễn ẩn song song cho mọi vị trí đầu vào và đầu ra. Trong các mô hình này, số phép toán cần thiết để liên kết tín hiệu từ hai vị trí đầu vào hoặc đầu ra bất kỳ tăng theo khoảng cách giữa các vị trí, tuyến tính đối với ConvS2S và logarit đối với ByteNet. Điều này làm cho việc học các phụ thuộc giữa các vị trí xa trở nên khó khăn hơn [12]. Trong Transformer, vấn đề này được giảm xuống còn một số phép toán hằng số, mặc dù phải trả giá bằng việc giảm độ phân giải thực tế do trung bình các vị trí được trọng số bởi chú ý, một hiệu ứng chúng tôi khắc phục bằng chú ý đa đầu như mô tả trong mục 3.2.

Tự chú ý, đôi khi gọi là intra-attention, là

Đa số các mô hình chuyển đổi dãy thần kinh cạnh tranh đều có cấu trúc mã-hóa và giải-mã [5, 2, 35]. Trong đó, bộ mã hóa ánh xạ một chuỗi đầu vào của biểu diễn ký hiệu ($x_1, \dots, x_n$) thành một chuỗi biểu diễn liên tục $\mathbf{z} = (z_1, \dots, z_n)$. Dựa trên $\mathbf{z}$, bộ giải mã sau đó tạo ra một chuỗi đầu ra ($y_1, \dots, y_m$) của ký hiệu từng phần tử một. Ở mỗi bước, mô hình này là hồi quy tự động [10], tiêu thụ các ký hiệu đã sinh ra trước đó như đầu vào bổ sung khi tạo ra ký hiệu tiếp theo.



Hình 1: Cấu trúc kiến trúc của mô hình Transformer.

Transformer tuân theo cấu trúc tổng thể này bằng cách sử dụng các lớp tự chú ý chồng lên nhau và các lớp kết nối đầy đủ, điểm, cho cả bộ mã hóa và bộ giải mã, được hiển thị ở nửa bên trái và phải của Hình 1, tương ứng.

3.1 Bộ Mã Hóa và Bộ Giải Mã

Bộ Mã Hóa: Bộ mã hóa bao gồm một chồng $N = 6$ lớp giống nhau. Mỗi lớp có hai lớp phụ. Lớp phụ thứ nhất là cơ chế tự chú ý đa đầu, và lớp phụ thứ hai là mạng nơ-ron đơn giản, kết nối đầy đủ, vị trí điểm. Chúng tôi sử dụng kết nối dư thừa [11] xung quanh mỗi lớp phụ, sau đó là chuẩn hóa lớp [1]. Đó là, đầu ra của mỗi lớp phụ là $\text{LayerNorm}(x + \text{Sublayer}(x))$, trong đó $\text{Sublayer}(x)$ là hàm được thực hiện bởi lớp phụ đó. Để hỗ trợ những kết nối dư thừa này, tất cả các lớp phụ trong mô hình, cũng như các lớp nhập, sản xuất đầu ra có chiều $d_{\text{model}} = 512$.

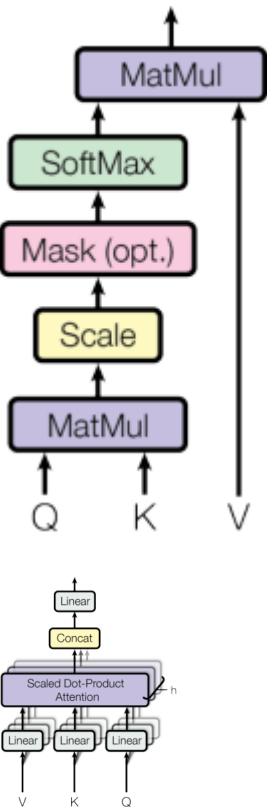
Bộ Giải Mã: Bộ giải mã cũng bao gồm một chồng $N = 6$ lớp giống nhau. Ngoài hai lớp phụ trong mỗi lớp bộ mã hóa, bộ giải mã thêm một lớp phụ thứ ba, thực hiện chú ý đa đầu trên đầu ra của chồng lớp bộ mã hóa. Tương tự như bộ mã hóa, chúng tôi sử dụng kết nối dư thừa xung quanh mỗi lớp phụ, sau đó là chuẩn hóa lớp. Chúng tôi cũng sửa đổi lớp phụ tự chú ý trong chồng lớp bộ giải mã để ngăn chặn vị trí không chú ý đến các vị trí sau đó. Sự che phủ này, kết hợp với thực tế rằng các nhập biểu diễn bị dịch một vị trí, đảm bảo rằng dự đoán cho vị trí i chỉ có thể phụ thuộc vào các đầu ra đã biết tại các vị trí nhỏ hơn i .

3.2 Cơ chế Chú Ý

Một hàm chú ý có thể được mô tả như ánh xạ một truy vấn và một tập hợp các cặp khóa-giá trị thành một đầu ra, trong đó truy vấn, khóa, giá trị và đầu ra đều là vector. Đầu ra được tính toán như một tổng trọng lượng

Chú ý tích vô hướng tỷ lệ

Chú ý đa đầu



Hình 2: (trái) Chú ý tích vô hướng tỷ lệ. (phải) Chú ý đa đầu bao gồm nhiều lớp chú ý chạy song song.

của các giá trị, trong đó trọng số được gán cho mỗi giá trị được tính toán bởi một hàm tương thích của truy vấn với khóa tương ứng.

3.2.1 Chú ý tích vô hướng tỷ lệ

Chúng tôi gọi chú ý cụ thể của chúng tôi là "chú ý tích vô hướng tỷ lệ" (Hình 2). Đầu vào bao gồm truy vấn và khóa có chiều dk , và giá trị có chiều dv . Chúng tôi tính toán tích vô hướng của truy vấn với tất cả các khóa, chia mỗi tích vô hướng đó cho $\sqrt{dk}$, và áp dụng hàm softmax để thu được trọng số trên các giá trị.

Trong thực tế, chúng tôi tính toán hàm chú ý trên một tập hợp các truy vấn đồng thời, đóng gói lại thành ma trận Q . Các khóa và giá trị cũng được đóng gói lại thành ma trận K và V . Chúng tôi tính toán ma trận đầu ra như:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Hai hàm chú ý được sử dụng phổ biến nhất là chú ý cộng thêm [2], và chú ý tích vô hướng (nhân). Chú ý tích vô hướng tương đương với thuật toán của chúng tôi, ngoại trừ hệ số tỷ lệ dk . Chú ý cộng thêm tính toán hàm tương thích sử dụng mạng nơ-ron kết nối đầy đủ với một lớp ẩn duy nhất. Trong khi hai phương pháp này tương tự về độ phức tạp lý thuyết, chú ý tích vô hướng nhanh hơn và tiết kiệm không gian hơn trong thực tế, vì nó có thể được thực hiện bằng mã nhân ma trận tối ưu hóa.

Trong trường hợp giá trị nhỏ của dk , hai cơ chế hoạt động tương tự, nhưng chú ý cộng thêm vượt trội so với chú ý tích vô hướng mà không có tỷ lệ cho giá trị lớn hơn của dk [3]. Chúng tôi nghi ngờ rằng đối với giá trị lớn của dk , các tích vô hướng tăng lớn về độ lớn, đẩy hàm softmax vào các vùng có gradient cực kỳ nhỏ. Để khắc phục tác động này, chúng tôi tỷ lệ các tích vô hướng bằng $\sqrt{dk}$.

3.2.2 Chú ý đa đầu

Thay vì thực hiện một hàm chú ý duy nhất với các khóa, giá trị và truy vấn có chiều d_{model} , chúng tôi tìm thấy lợi ích khi tuyến tính hóa các truy vấn, khóa và giá trị h lần với các phép chiếu học khác nhau đến dk , dk và dv chiều, tương ứng. Trên mỗi phiên bản đã chiếu của truy vấn, khóa và giá trị, chúng tôi sau đó thực hiện hàm chú ý song song, dẫn đến các giá trị đầu ra có chiều dv .

Chú ý đa đầu cho phép mô hình cùng lúc chú ý đến thông tin từ các không gian biểu diễn phụ khác nhau ở các vị trí khác nhau. Với một đầu chú ý duy nhất, trung bình hóa ngăn chặn điều này.

 tất cả các giá trị trong đầu vào của hàm softmax tương ứng với các kết nối không hợp lệ. Xem Hình 2.

3.3 Mạng Nơ-ron Phân Loại theo Vị Trí

Ngoài các lớp phụ thuộc chú ý, mỗi lớp trong bộ mã hóa và giải mã của chúng tôi chứa một mạng nơ-ron phân loại đầy đủ kết nối, được áp dụng riêng biệt và đồng nhất cho mỗi vị trí. Điều này bao gồm hai biến đổi tuyến tính với hàm kích hoạt ReLU giữa chúng.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

Trong khi các biến đổi tuyến tính giống nhau qua các vị trí khác nhau, chúng sử dụng các tham số khác nhau từ lớp này sang lớp khác. Một cách khác để mô tả điều này là hai phép tích chập với kích thước hạt 1. Chiều của đầu vào và đầu ra là $d_{\text{model}} = 512$, và lớp giữa có chiều $d_{\text{ff}} = 2048$.

3.4 Đánh dấu và Hàm softmax

Tương tự như các mô hình chuyển đổi chuỗi khác, chúng tôi sử dụng các đánh dấu học được để chuyển đổi các token đầu vào và token đầu ra thành các vector có chiều d_{model} . Chúng tôi cũng sử dụng phép biến đổi tuyến tính học được và hàm softmax để chuyển đổi đầu ra của giải mã thành xác suất token tiếp theo dự đoán. Trong mô hình của chúng tôi, chúng tôi chia sẻ cùng một ma trận trọng số giữa hai lớp đánh dấu và phép biến đổi tuyến tính trước hàm softmax, tương tự như [30]. Trong các lớp đánh dấu, chúng tôi nhân những trọng số đó với $\sqrt{d_{\text{model}}}$.

Bảng 1: Độ dài đường đi tối đa, độ phức tạp từng lớp và số lượng tối thiểu các phép toán tuần tự cho các loại lớp khác nhau. n là độ dài chuỗi, d là chiều biểu diễn, k là kích thước hạt của các phép tích chập và r là kích thước khu vực trong tự chú ý bị hạn chế.

Loại lớp	Độ phức tạp từng lớp	Phép toán tuần tự	Độ dài đường đi tối đa
Tự chú ý	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Mạng nơ-ron hồi quy	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Mạng tích chập	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log k(n))$
Tự chú ý (bị hạn chế)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 Mã hóa Vị Trí

Vì mô hình của chúng tôi không chứa hồi quy hoặc tích chập, để cho mô hình có thể sử dụng thứ tự của chuỗi, chúng tôi phải thêm một số thông tin về vị trí tương đối hoặc tuyệt đối của các token trong chuỗi. Để đạt được điều này, chúng tôi thêm "mã hóa vị trí" vào các đánh dấu đầu vào ở đáy của bộ mã hóa và giải mã. Các mã hóa vị trí có cùng chiều d_{model} như các đánh dấu, vì vậy chúng có thể được cộng lại. Có nhiều lựa chọn cho mã hóa vị trí, học được và cố định [9].

Trong công trình này, chúng tôi sử dụng các hàm sin và cos của các tần số khác nhau:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

trong đó pos là vị trí và i là chiều. Đó là, mỗi chiều của mã hóa vị trí tương ứng với một sóng sin. Các bước sóng tạo thành một cấp số nhân từ 2π đến $10000 \cdot 2\pi$. Chúng tôi chọn hàm này vì chúng tôi giả định rằng nó sẽ cho phép mô hình dễ dàng học để tập trung vào vị trí tương đối, vì cho bất kỳ offset cố định k , PE_{pos+k} có thể được biểu diễn như một hàm tuyến tính của PE_{pos} .

Chúng tôi cũng đã thử nghiệm với việc sử dụng các mã hóa vị trí học được [9] thay vào đó, và phát hiện ra rằng hai phiên bản sản xuất kết quả gần như giống hệt nhau (xem Bảng 3 dòng (E)). Chúng tôi chọn phiên bản $\sin\text{-}\cos$ vì nó có thể cho phép mô hình extrapolate đến độ dài chuỗi dài hơn những gì gặp phải trong quá trình huấn luyện.

4 Tại Sao Tự Chú Ý

Trong phần này, chúng ta so sánh các khía cạnh khác nhau của lớp tự chú ý với các lớp hồi quy và tích phân thường được sử dụng để ánh xạ một chuỗi biểu diễn ký hiệu có độ dài biến đổi ($x_1, \dots, x_n$) sang một chuỗi khác có cùng độ dài ($z_1, \dots, z_n$), với $x_i, z_i \in \mathbb{R}^d$, như một lớp ẩn trong một bộ mã hóa hoặc giải mã chuyển đổi chuỗi diễn hình. Để thúc đẩy việc sử dụng tự chú ý, chúng tôi xem xét ba yêu cầu.

Một là tổng độ phức tạp tính toán trên mỗi lớp. Hai là lượng tính toán có thể song song hóa, được đo bằng số lượng tối thiểu các phép tính tuần tự cần thiết.

Yêu cầu thứ ba là chiều dài đường giữa các mối quan hệ phụ thuộc xa trong mạng. Học các mối quan hệ phụ thuộc xa là thách thức chính trong nhiều tác vụ chuyển đổi chuỗi. Một yếu tố ảnh hưởng đến khả năng học các mối quan hệ phụ thuộc xa là chiều dài của các đường đi từ trước và sau tín hiệu phải đi qua trong mạng. Khi các đường này ngắn hơn giữa bất kỳ kết hợp nào của vị trí trong chuỗi đầu vào và đầu ra, việc học các mối quan hệ phụ thuộc xa trở nên dễ dàng hơn [12]. Vì vậy, chúng tôi cũng so sánh chiều dài đường lớn nhất giữa hai vị trí đầu vào và đầu ra trong các mạng được tạo thành từ các loại lớp khác nhau.

Như được ghi chú trong bảng 1, một lớp tự chú ý kết nối tất cả các vị trí với một số lượng cố định các phép tính tuần tự được thực hiện, trong khi một lớp hồi quy yêu cầu $O(n)$ phép tính tuần tự. Về độ phức tạp tính toán, các lớp tự chú ý nhanh hơn các lớp hồi quy khi độ dài chuỗi n nhỏ hơn chiều dài biểu diễn d , điều này thường xảy ra với các biểu diễn câu được sử dụng bởi các mô hình tiên tiến trong dịch máy, như word-piece [38] và byte-pair [31] biểu diễn. Để cải thiện hiệu suất tính toán cho các tác vụ liên quan đến chuỗi rất dài, tự chú ý có thể bị giới hạn chỉ xem xét một khu vực gần với vị trí đầu ra tương ứng trong chuỗi đầu vào. Điều này sẽ tăng chiều dài đường lớn nhất lên $O(n/r)$. Chúng tôi dự định nghiên cứu tiếp cận này thêm trong công việc tương lai.

Một lớp tích phân đơn với chiều rộng hạt $k < n$ không kết nối tất cả các cặp vị trí đầu vào và đầu ra. Việc làm điều này yêu cầu một chồng $O(n/k)$ lớp tích phân trong trường hợp hạt liền kề, hoặc $O(\log_k(n))$ trong trường hợp tích phân giãn nở [18], tăng chiều dài của đường dài nhất giữa bất kỳ hai vị trí nào trong mạng. Các lớp tích phân thường đắt hơn các lớp hồi quy, với một yếu tố k . Tuy nhiên, các lớp tích phân tách biệt [6] giảm đáng kể độ phức tạp, xuống $O(k \cdot n \cdot d + n \cdot d^2)$. Ngay cả khi $k = n$, độ phức tạp của một lớp tích phân tách biệt vẫn bằng sự kết hợp của một lớp tự chú ý và một lớp phát triển điểm, cách tiếp cận mà chúng tôi sử dụng trong mô hình của mình.

Như lợi ích phụ, tự chú ý có thể tạo ra các mô hình dễ hiểu hơn. Chúng tôi kiểm tra các phân phối chú ý từ các mô hình của mình và trình bày và thảo luận các ví dụ trong phụ lục. Không chỉ các đầu chú ý riêng lẻ rõ ràng học để thực hiện các nhiệm vụ khác nhau, mà nhiều trong số chúng xuất hiện có hành vi liên quan đến cấu trúc cú pháp và ngữ nghĩa của các câu.

5 Huấn luyện

Phần này mô tả chế độ huấn luyện cho các mô hình của chúng tôi.

5.1 Dữ liệu huấn luyện và Batch

Chúng tôi huấn luyện trên tập dữ liệu chuẩn WMT 2014 tiếng Anh-Đức bao gồm khoảng 4,5 triệu cặp câu. Câu được mã hóa bằng byte-pair encoding [3], có từ vựng nguồn-tujuan chung khoảng 37000 token. Đối với tiếng Anh-Tiếng Pháp, chúng tôi đã sử dụng tập dữ liệu WMT 2014 tiếng Anh-Tiếng Pháp lớn hơn nhiều, bao gồm 36 triệu câu và chia token thành một từ vựng word-piece 32000 từ [38]. Các cặp câu được nhóm lại theo độ dài chuỗi gần đúng. Mỗi batch huấn luyện chứa một tập các cặp câu chứa khoảng 25000 token nguồn và 25000 token mục tiêu.

5.2 Thiết bị và Lịch

Chúng tôi huấn luyện các mô hình của mình trên một máy với 8 GPU NVIDIA P100. Đối với các mô hình cơ bản sử dụng siêu tham số được mô tả trong toàn bộ bài báo, mỗi bước huấn luyện mất khoảng 0,4 giây. Chúng tôi huấn luyện các mô hình cơ bản trong tổng cộng 100.000 bước hoặc 12 giờ. Đối với các mô hình lớn (mô tả ở hàng dưới cùng của bảng 3), thời gian mỗi bước là 1,0 giây. Các mô hình lớn được huấn luyện trong 300.000 bước (3,5 ngày).

5.3 Tối ưu hóa

Chúng tôi sử dụng tối ưu hóa Adam [20] với $\beta_1 = 0,9$, $\beta_2 = 0,98$ và $\epsilon = 10^{-9}$. Chúng tôi thay đổi tốc độ học trong suốt quá trình huấn luyện, theo công thức:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

Điều này tương ứng với việc tăng tốc độ học tuyến tính trong *warmup* bước đầu tiên và giảm nó sau đó tỷ lệ thuận với nghịch đảo bình phương của số bước. Chúng tôi sử dụng *warmup* = 4000 bước.

5.4 Quy Hoạch

Chúng tôi áp dụng ba loại quy tắc hóa trong quá trình huấn luyện:

Bảng 2: Transformer đạt được điểm số BLEU tốt hơn so với các mô hình hàng đầu trước đó trên bài kiểm tra mới test2014 từ tiếng Anh sang tiếng Đức và từ tiếng Anh sang tiếng Pháp với chi phí huấn luyện thấp hơn đáng kể.

[[BL EU Chi phí huấn luyện (FLOPs) --- --- --- --- --- Mô hình]]]] EN-DE EN-FR EN-DE EN-FR ByteNet [18]					
23.75]]]] Deep-Att + PosUnk [39] 39.2 1_0 · 10 ²⁰					
GNMT + RL [38] 24.6 39.92 2.3 · 10 ¹⁹ 1.4 · 10 ²⁰	ConvS2S [9] 25.16 40.46 9.6 · 10 ¹⁸ 1.5 · 10 ²⁰	MoE[32] 26.03			
40.56 2.0 · 10 ¹⁹ 1.2 · 10 ²⁰	Deep-Att + PosUnk Ensemble [39] 40.4 8.0 · 10 ²⁰	GNMT + RL Ensemble [38] 26.30			
41.16 1.8 · 10 ²⁰ 1.1 · 10 ²¹	ConvS2S Ensemble[9] 26.36 41.29 7.7 · 10 ¹⁹ 1.2 · 10 ²¹	Transformer (mô hình cơ bản)			
27.3 38.1 3 ^{**} · 3 ^{****} · ** 10 ¹⁸	Transformer (lớn) 28.4 41.8 2.3_10 ¹⁹				

Quy tắc hóa dư thừa Chúng tôi áp dụng quy tắc hóa [33] cho kết quả của mỗi lớp phụ trước khi thêm vào đầu vào lớp phụ và chuẩn hóa. Ngoài ra, chúng tôi áp dụng quy tắc hóa cho tổng của các biểu diễn và mã hóa vị trí trong cả chồng encoder và decoder. Đối với mô hình cơ bản, chúng tôi sử dụng tỷ lệ $P_{drop} = 0.1$.

Trừ tròn nhân Trong quá trình huấn luyện, chúng tôi áp dụng trừ tròn nhân có giá trị $\epsilon/s = 0.1$ [36]. Điều này làm giảm độ phức tạp, vì mô hình học cách trở nên không chắc chắn hơn, nhưng cải thiện độ chính xác và điểm số BLEU.

6 Kết quả

6.1 Dịch máy

Trên nhiệm vụ dịch từ tiếng Anh sang tiếng Đức của WMT 2014, mô hình lớn của Transformer (Transformer (lớn) trong Bảng 2) vượt trội hơn các mô hình đã báo cáo trước đây (bao gồm cả tập hợp) hơn 2.0 BLEU, thiết lập một điểm số BLEU hàng đầu mới là 28.4 . Cấu hình của mô hình này được liệt kê ở dòng cuối cùng của Bảng 3. Thời gian huấn luyện mất 3.5 ngày trên 8 GPU P100. Ngay cả mô hình cơ bản cũng vượt trội hơn tất cả các mô hình và tập hợp đã công bố trước đây, với chi phí huấn luyện thấp hơn đáng kể so với bất kỳ mô hình cạnh tranh nào.

Trên nhiệm vụ dịch từ tiếng Anh sang tiếng Pháp của WMT 2014, mô hình lớn của chúng tôi đạt được điểm số BLEU là 41.0 , vượt trội hơn tất cả các mô hình đơn đã công bố trước đây, với chi phí huấn luyện thấp hơn khoảng $1/4$ so với mô hình hàng đầu trước đây. Mô hình Transformer (lớn) được huấn luyện cho dịch từ tiếng Anh sang tiếng Pháp sử dụng tỷ lệ quy tắc hóa $P_{drop} = 0.1$ thay vì 0.3 .

Đối với các mô hình cơ bản, chúng tôi sử dụng một mô hình duy nhất được thu được bằng cách trung bình 5 điểm kiểm tra cuối cùng, được viết tại khoảng cách 10 phút. Đối với các mô hình lớn, chúng tôi trung bình 20 điểm kiểm tra cuối cùng. Chúng tôi sử dụng thuật toán tìm kiếm chiều rộng với kích thước chiều rộng là 4 và hệ số phạt độ dài $\alpha = 0.6$ [38]. Các siêu tham số này được chọn sau khi thử nghiệm trên tập phát triển. Chúng tôi đặt độ dài tối đa của đầu ra trong quá trình suy luận là độ dài đầu vào + 50, nhưng kết thúc sớm nếu có thể [38].

Bảng 2 tóm tắt kết quả của chúng tôi và so sánh chất lượng dịch và chi phí huấn luyện của chúng tôi với các kiến trúc mô hình khác từ văn獻. Chúng tôi ước tính số lượng phép tính dấu phẩy động được sử dụng để huấn luyện một mô hình bằng cách nhân thời gian huấn luyện, số lượng GPU được sử dụng, và một ước tính về năng lực dấu phẩy động đơn chính xác duy trì của mỗi GPU⁵.

6.2 Các biến thể của mô hình

Để đánh giá tầm quan trọng của các thành phần khác nhau của Transformer, chúng tôi đã thay đổi mô hình cơ bản theo nhiều cách khác nhau, đo lường sự thay đổi trong hiệu suất trên tập phát triển newstest2013 từ tiếng Anh sang tiếng Đức.

⁵Chúng tôi sử dụng giá trị 2.8, 3.7, 6.0 và 9.5 TFLOPS cho K80, K40, M40 và P100 tương ứng.

Bảng 3: Biến thể của kiến trúc Transformer. Các giá trị không được liệt kê giống như những giá trị của mô hình cơ bản. Tất cả các chỉ số đều trên tập phát triển newstest2013 từ tiếng Anh sang tiếng Đức. Các độ phức tạp được liệt kê theo từng từ-piece, theo mã hóa byte-pair của chúng tôi, và không nên so sánh với độ phức tạp theo từng từ.

	N	$_{d_model}$	$_{d_ff}$	h	dk	dv	P_{drop}	ϵ/s	bước huấn luyện	PPL (dev)	BLE U (dev)	tham số _x_1 06
--	-----	---------------	------------	-----	------	------	------------	--------------	-----------------------	--------------	-------------------	--------------------------

mô hình cơ bản	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
				1	512	512				5.29	24.9	
A				4	128	128				5.00	25.5	
()				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
					16					5.16	25.	

Để đánh giá khả năng tổng quát của Transformer đối với các tác vụ khác, chúng tôi đã thực hiện các thí nghiệm trên phân tích cú pháp cấu trúc của tiếng Anh. Tác vụ này mang lại những thách thức cụ thể: kết quả đầu ra chịu sự ràng buộc mạnh mẽ về mặt cấu trúc và dài hơn nhiều so với đầu vào. Hơn nữa, các mô hình chuỗi đến chuỗi dựa trên mạng nơ-ron hồi quy (RNN) chưa thể đạt được kết quả tốt nhất trong các chế độ dữ liệu nhỏ [37].

Chúng tôi đã huấn luyện một Transformer 4 lớp với $d_{model} = 1024$ trên phần Wall Street Journal (WSJ) của Penn Treebank [25], khoảng 40K câu huấn luyện. Chúng tôi cũng huấn luyện nó trong một môi trường bán giám sát, sử dụng tập dữ liệu lớn hơn từ BerkleyParser với khoảng 17 triệu câu [37]. Chúng tôi sử dụng từ điển gồm 16K token cho trường hợp chỉ WSJ và từ điển gồm 32K token cho trường hợp bán giám sát.

Chúng tôi đã thực hiện một số lượng nhỏ thí nghiệm để chọn xác suất bỏ sót (dropout), cả cơ chế chú ý và dư (residual) (mục 5.4), tốc độ học và kích thước beam trên tập phát triển Section 22, tất cả các tham số khác đều không thay đổi so với mô hình dịch từ tiếng Anh sang tiếng Đức cơ bản. Trong quá trình suy luận, chúng tôi

Bảng 4: Transformer tổng quát hóa tốt cho phân tích cú pháp cấu trúc của tiếng Anh (Kết quả trên Section 23 của WSJ)

Phân tích cú pháp	Huấn luyện	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	Chỉ WSJ, phân biệt	88.3
Petrov et al. (2006) [29]	Chỉ WSJ, phân biệt	90.4
Zhu et al. (2013) [40]	Chỉ WSJ, phân biệt	90.4
Dyer et al. (2016) [8]	Chỉ WSJ, phân biệt	91.7
Transformer (4 lớp)	Chỉ WSJ, phân biệt	91.3
Zhu et al. (2013) [40]	Bán giám sát	91.3
Huang & Harper (2009) [14]	Bán giám sát	91.3
McClosky et al. (2006) [26]	Bán giám sát	92.1
Vinyals & Kaiser et al. (2014) [37]	Bán giám sát	92.1

Transformer (4 lớp)	Bán giám sát	92.7
Luong et al. (2015) [23]	Tác vụ đa nhiệm	93.0
Dyer et al. (2016) [8]	Tạo ra	93.3

chúng tôi tăng chiều dài tối đa của đầu ra lên đầu vào + 300. Chúng tôi sử dụng kích thước beam là 21 và $\alpha = 0.3$ cho cả trường hợp chỉ WSJ và bán giám sát.

Kết quả của chúng tôi trong Bảng 4 cho thấy mặc dù thiếu điều chỉnh cụ thể cho tác vụ, mô hình của chúng tôi hoạt động tốt hơn mong đợi, đạt được kết quả tốt hơn tất cả các mô hình trước đây báo cáo ngoại trừ mạng ngữ pháp nơ-ron hồi quy [8].

Trái ngược với các mô hình chuỗi đến chuỗi dựa trên mạng nơ-ron hồi quy [37], Transformer vượt trội so với BerkleyParser [29] ngay cả khi huấn luyện chỉ trên tập huấn luyện WSJ của 40K câu.

7 Kết luận

Trong công trình này, chúng tôi giới thiệu Transformer, mô hình chuyển đổi chuỗi đầu tiên dựa hoàn toàn trên cơ chế chú ý, thay thế các lớp hồi quy thường được sử dụng trong kiến trúc bộ mã hóa - bộ giải mã bằng chú ý đa đầu.

Đối với các tác vụ dịch, Transformer có thể được huấn luyện nhanh hơn đáng kể so với các kiến trúc dựa trên mạng nơ-ron hồi quy hoặc lớp tích chập. Trên cả tác vụ dịch WMT 2014 tiếng Anh sang tiếng Đức và WMT 2014 tiếng Anh sang tiếng Pháp, chúng tôi đạt được trạng thái tốt nhất mới. Trong tác vụ trước, mô hình tốt nhất của chúng tôi vượt trội so với tất cả các nhóm mô hình trước đây.

Chúng tôi rất hào hứng về tương lai của các mô hình dựa trên cơ chế chú ý và kế hoạch áp dụng chúng vào các tác vụ khác. Chúng tôi kế hoạch mở rộng Transformer cho các vấn đề liên quan đến các mô hình đầu vào và đầu ra khác ngoài văn bản và nghiên cứu các cơ chế chú ý cục bộ, bị hạn chế để xử lý hiệu quả các đầu vào và đầu ra lớn như hình ảnh, âm thanh và video. Làm cho việc tạo ra ít chuỗi hơn là một mục tiêu nghiên cứu khác của chúng tôi.

Mã nguồn chúng tôi sử dụng để huấn luyện và đánh giá các mô hình của chúng tôi có sẵn tại <https://github.com/tensorflow/tensor2tensor>.

Tri ân Chúng tôi cảm ơn Nal Kalchbrenner và Stephan Gouws vì những nhận xét, sửa đổi và nguồn cảm hứng hữu ích của họ.

References [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR* , abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357* , 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR* , abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL* , 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2* , 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850* , 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* , pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation* , 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing* , pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410* , 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)* , 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)* , 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2* , 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations* , 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR* , 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722* , 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130* , 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence

to sequence learning. *arXiv preprint arXiv:1511.06114* , 2015.

- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attentionbased neural machine translation. *arXiv preprint arXiv:1508.04025* , 2015.

11

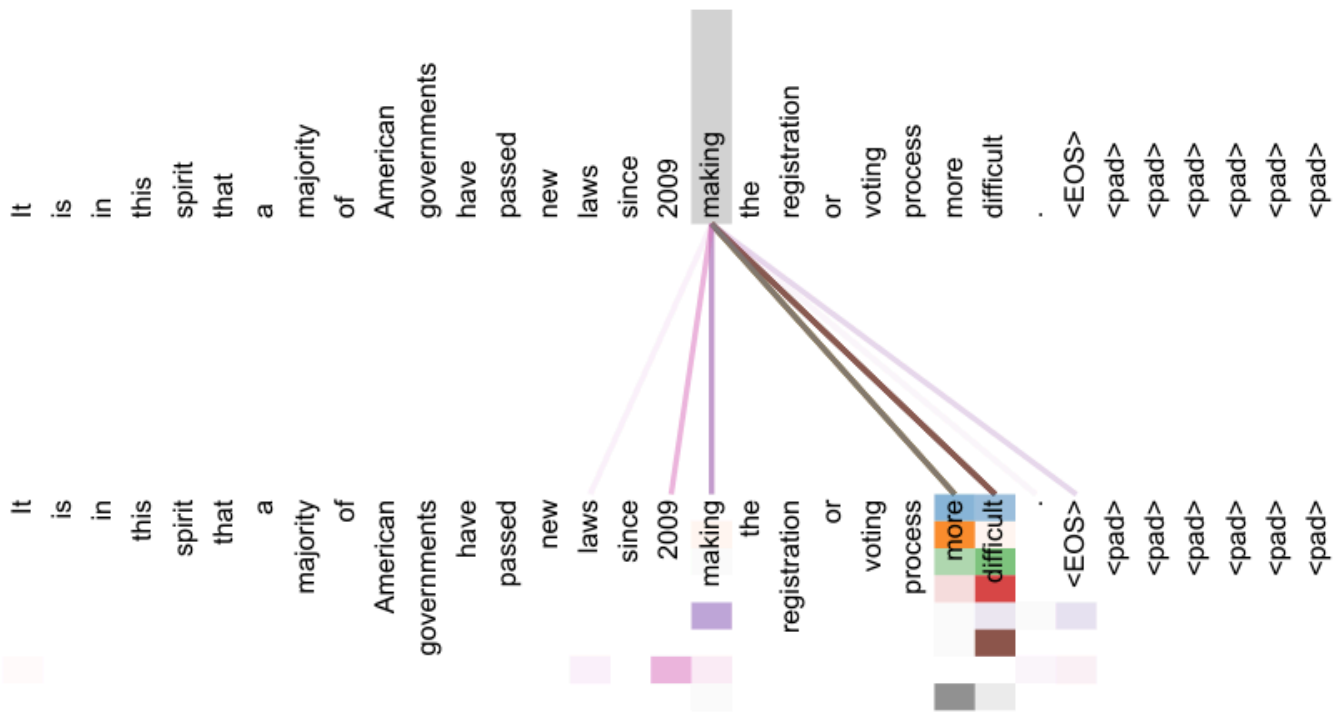
- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics* , 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference* , pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing* , 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304* , 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL* , pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859* , 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909* , 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* , 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* , 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28* , pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems* , pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR* , abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems* , 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* , 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward

connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

12

Attention Visualizations

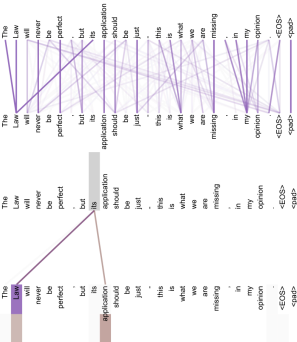


It is in this spirit that a majority of American governments have passed new laws since 2009 making the registration or voting process more difficult .

It is in this spirit that a majority of American governments have passed new laws since 2009 making the registration or voting process more difficult .

Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color.

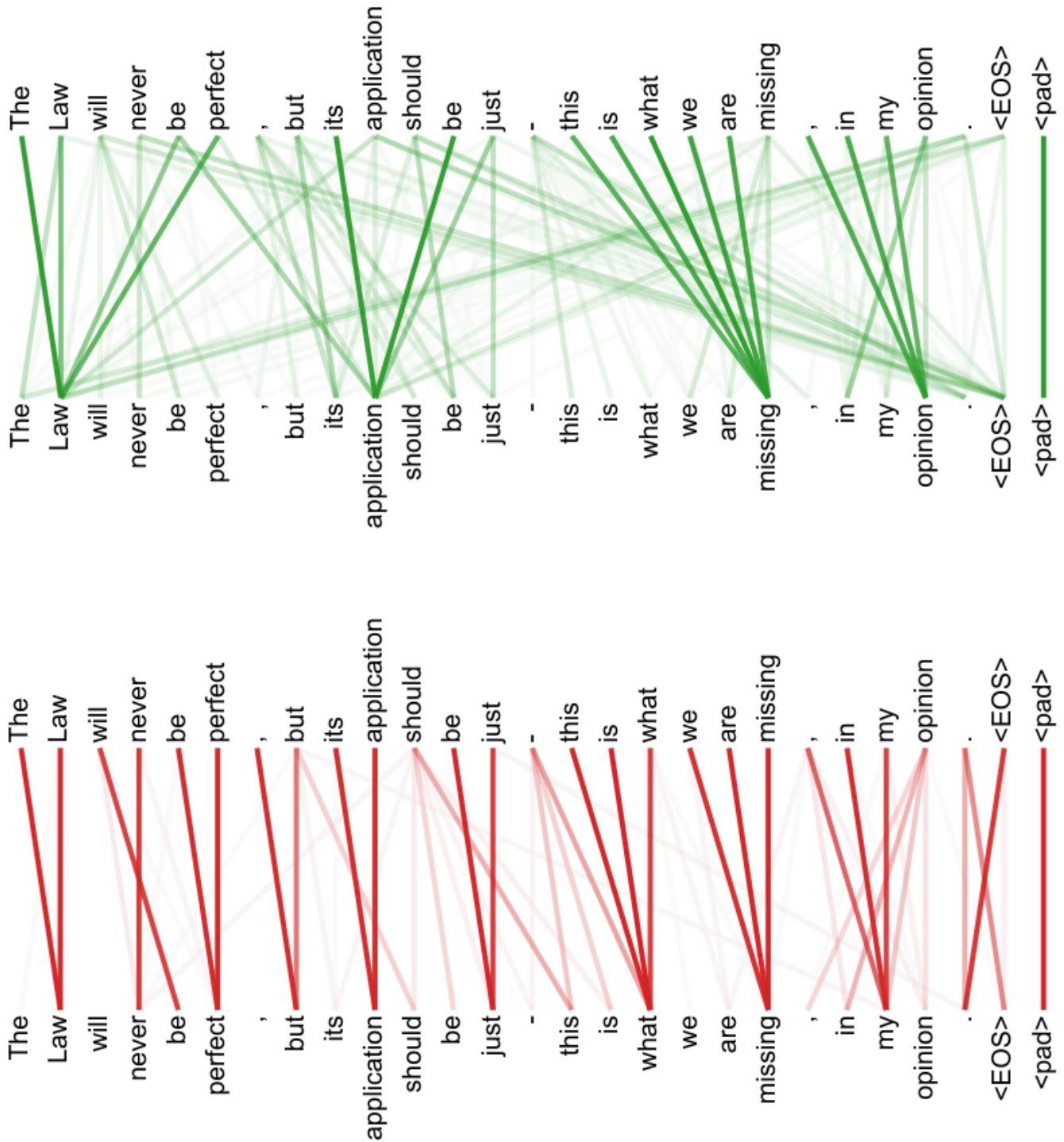
13



The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .
The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .
The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .
The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .

Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

14



The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .

The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .
The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .
The Law will never be perfect , but its application should be just - this is what we are missing , in my opinion .

Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

